

The background is a dark gray field filled with a complex network of thin, dark gray lines. These lines connect various colored circular nodes. The nodes are in shades of red, green, blue, and black, with some appearing larger than others. The network is dense and somewhat chaotic, with many lines crossing each other. There are also many small, dark gray dots scattered across the background, giving it a starry or data-like appearance.

Autonomous Agents

Tools and Model Context Protocol

MCP Definition

- Open standard introduced by **Anthropic**
- Standardizes how LLMs connect to:
 - External tools
 - Databases
 - APIs
 - File systems
- Eliminates custom integrations for every application
- **Key Idea:** One common protocol for connecting models to external capabilities.



Why Do We Need MCP?

Without MCP

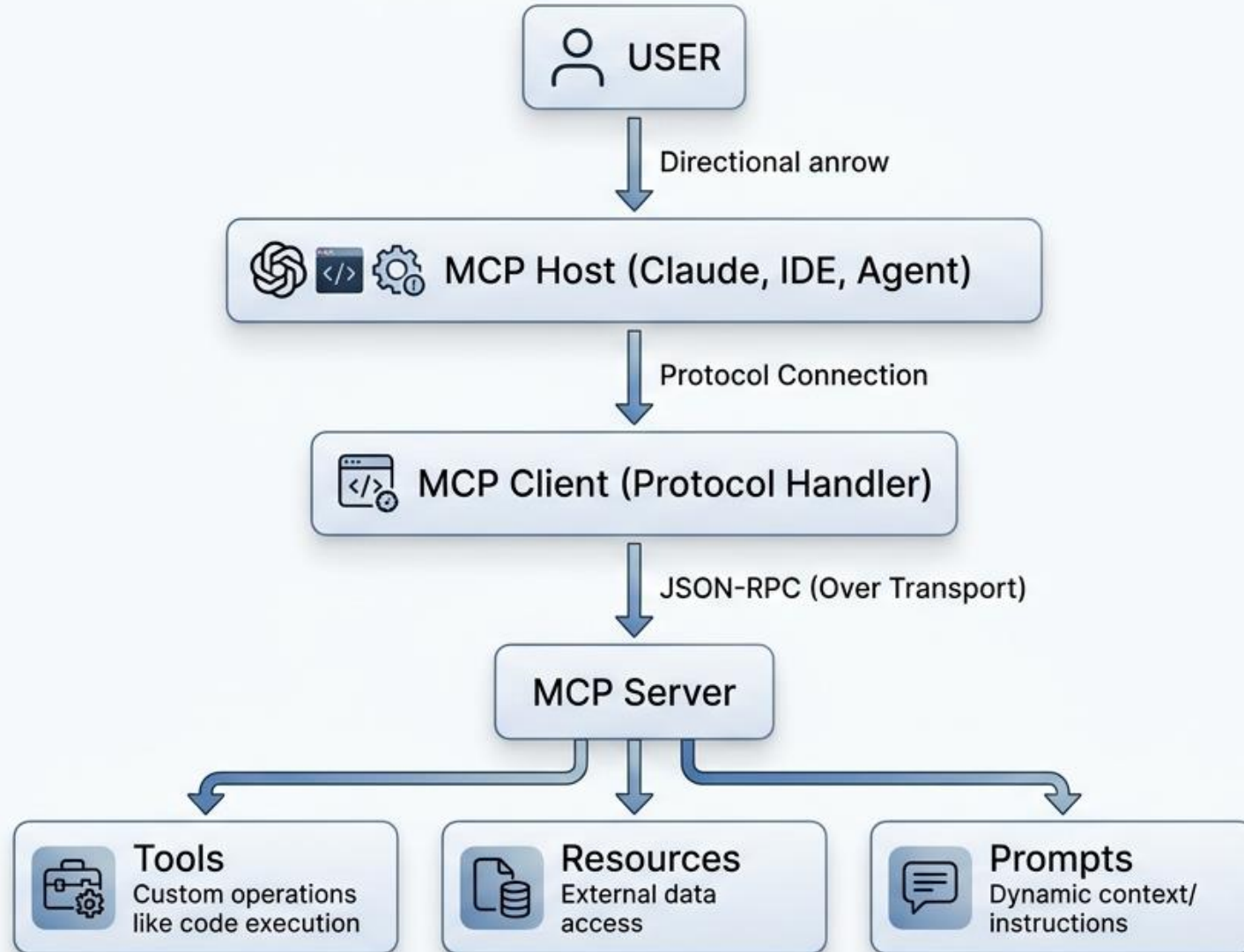
- Every agent needs custom integrations
- Different APIs and interfaces for every system
- High development and maintenance cost

With MCP

- One standardized interface
- Reusable integrations
- Any MCP-compatible client can communicate with any MCP server

Analogy: MCP is to AI tools what **USB-C** is to computer peripherals.

MCP Architecture Overview



MCP Components

MCP Host

The application users interact with

Examples:

- Claude
- IDEs
- Custom AI agents

MCP Client

Runs inside the host

Maintains a **1:1 connection** with an MCP server

Performs protocol handshake and communication

MCP Server

Lightweight service exposing capabilities

Can be shared across many clients

Capabilities Exposed by an MCP Server

Tools

- Functions the model can execute

Examples:

- `get_customer_by_email`
- `create_order`

Capabilities Exposed by an MCP Server

Resources

- Read-only information

Examples:

- Documents Files
- Database
- records

Capabilities Exposed by an MCP Server

Prompts

- Reusable prompt templates

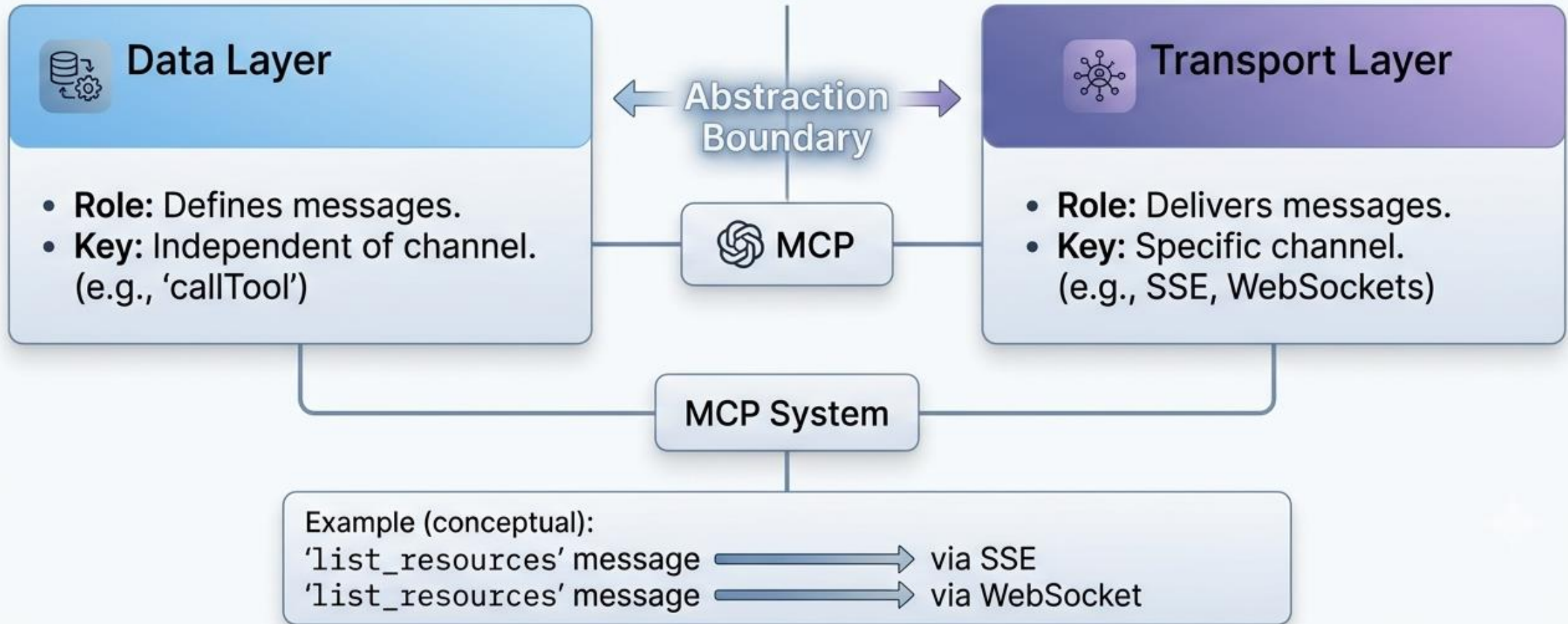
Examples:

- Report generation
- Code review templates
- Data analysis prompts

MCP Design Separation

Role: Defines what messages mean.

Role: Defines how messages travel.



The Data Layer

The data layer is a **JSON-RPC 2.0** protocol that defines:

Message structure

Message semantics

Client-server negotiation

Capability discovery

Tool execution

Data Layer Responsibilities

Lifecycle Management

- Initialize connections
- Negotiate capabilities
- Terminate connections

Server Features

- Tools
- Resources
- Prompts

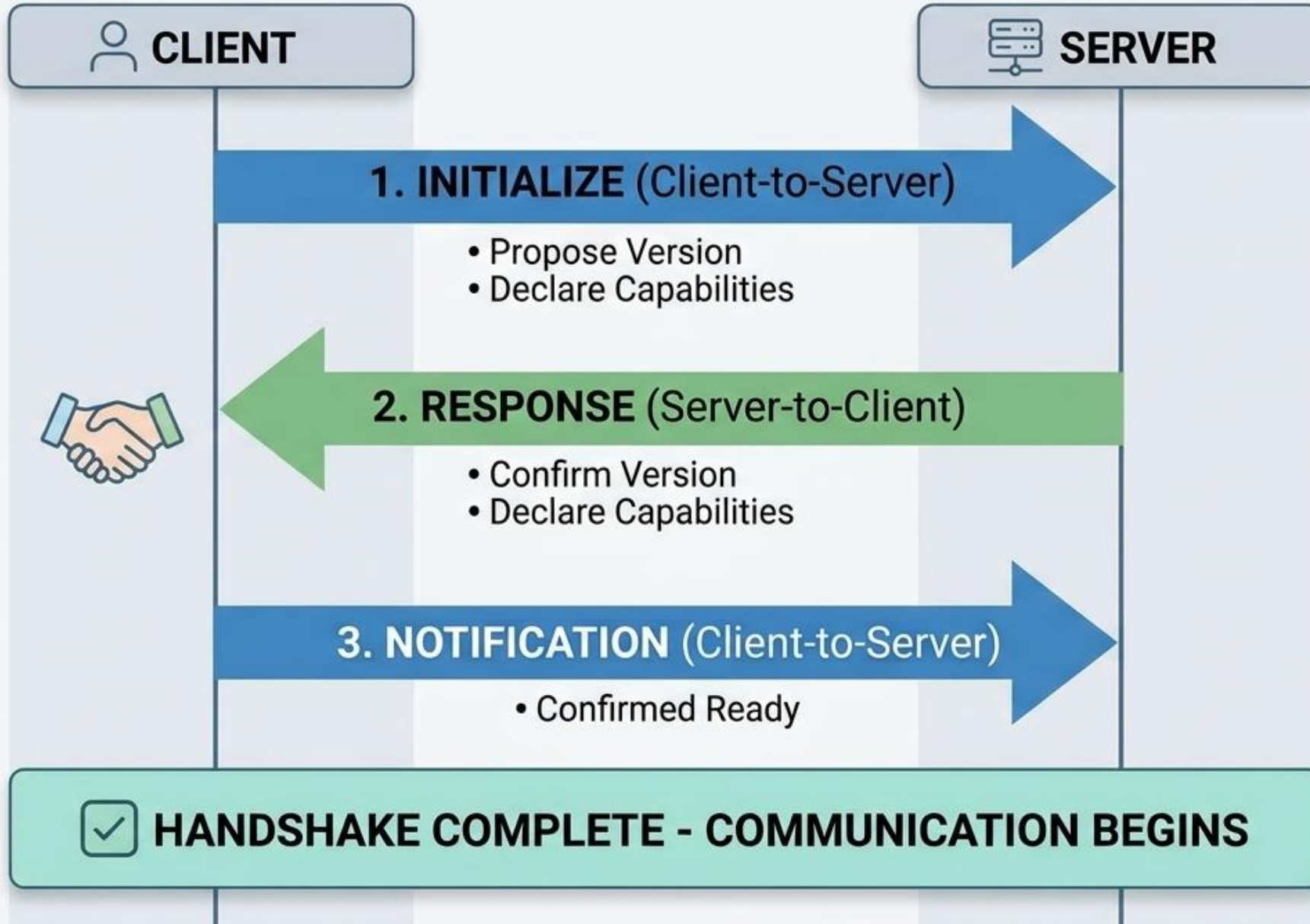
Client Features

- Sampling
- Elicitation
- Logging

Utility Features

- Notifications
- Progress tracking

MCP 2.0 Simplified Lifecycle Handshake



Lifecycle Handshake Communication

// 1. Client -> Server: initialize

```
{  
  "jsonrpc": "2.0", "id": 1, "method": "initialize",  
  "params": {  
    "protocolVersion": "2025-06-18",  
    "capabilities": { "elicitation": {} },  
    "clientInfo": { "name": "example-client", "version": "1.0.0" }  
  }  
}
```

// 2. Server -> Client: capabilities it supports

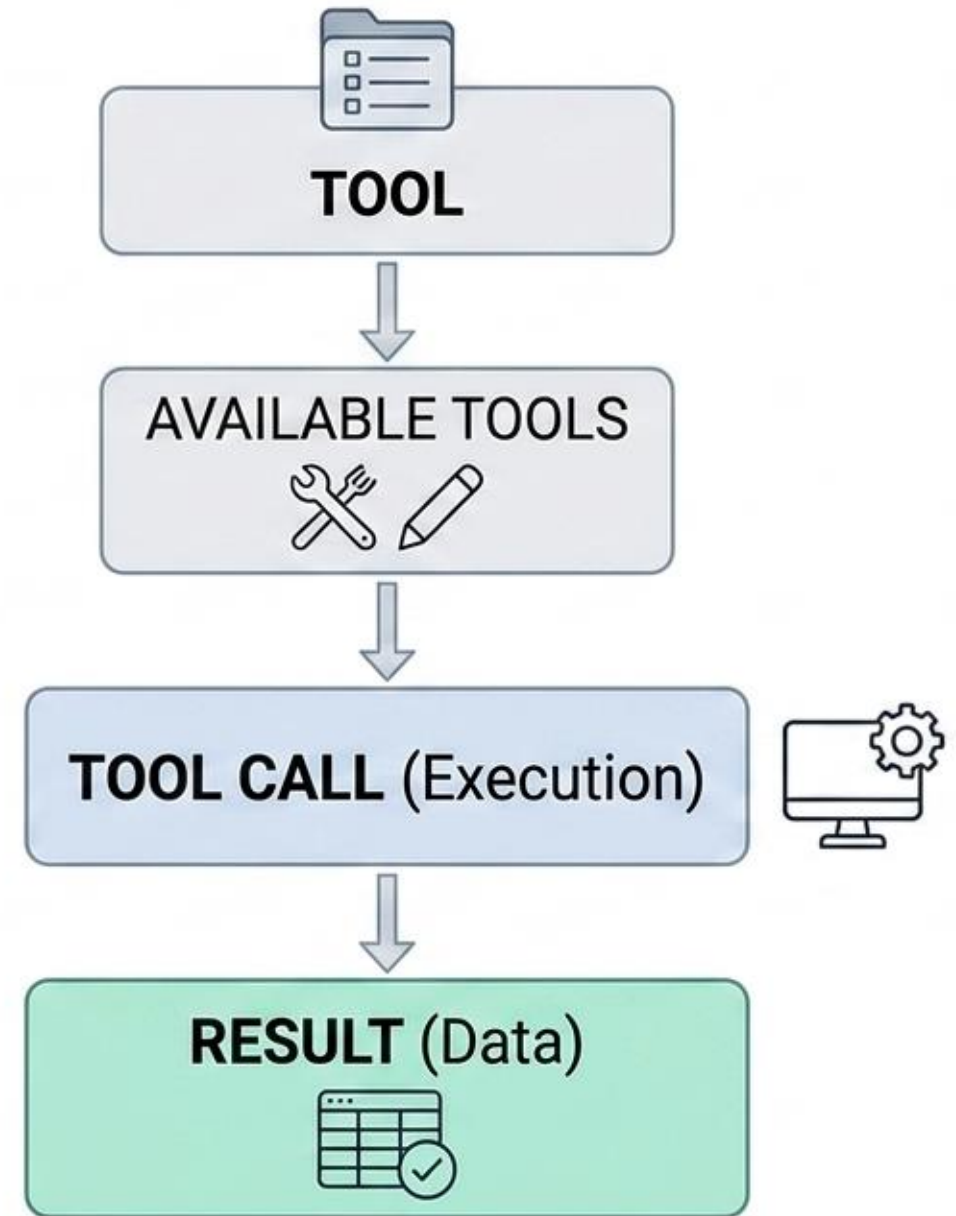
```
{  
  "jsonrpc": "2.0", "id": 1,  
  "result": {  
    "protocolVersion": "2025-06-18",  
    "capabilities": { "tools": { "listChanged": true }, "resources": {} },  
    "serverInfo": { "name": "brewcraft", "version": "1.0.0" }  
  }  
}
```

// 3. Client -> Server: ready (a notification — no "id", no response expected)

```
{ "jsonrpc": "2.0", "method": "notifications/initialized" }
```

Discovery & Tool Execution

- Clients never assume what a server supports.
- Instead they follow a standard workflow:
- Benefits:
 - No hardcoded tool definitions
 - Dynamic discovery
 - Extensible servers



Discovery Code

// Client -> Server: what tools do you have?

```
{ "jsonrpc": "2.0", "id": 2, "method": "tools/list" }
```

// Server -> Client: here's the schema for each one

```
{  
  "jsonrpc": "2.0", "id": 2,  
  "result": {  
    "tools": [{  
      "name": "get_customer",  
      "description": "Look up a BrewCraft customer by email address.",  
      "inputSchema": { "type": "object", "properties": { "email": { "type": "string" } }, "required": [ "email" ] }  
    }  
  ]  
}  
}
```

// Client -> Server: execute it

```
{  
  "jsonrpc": "2.0", "id": 3, "method": "tools/call",  
  "params": { "name": "get_customer", "arguments": { "email": "sam@example.com" } }  
}
```

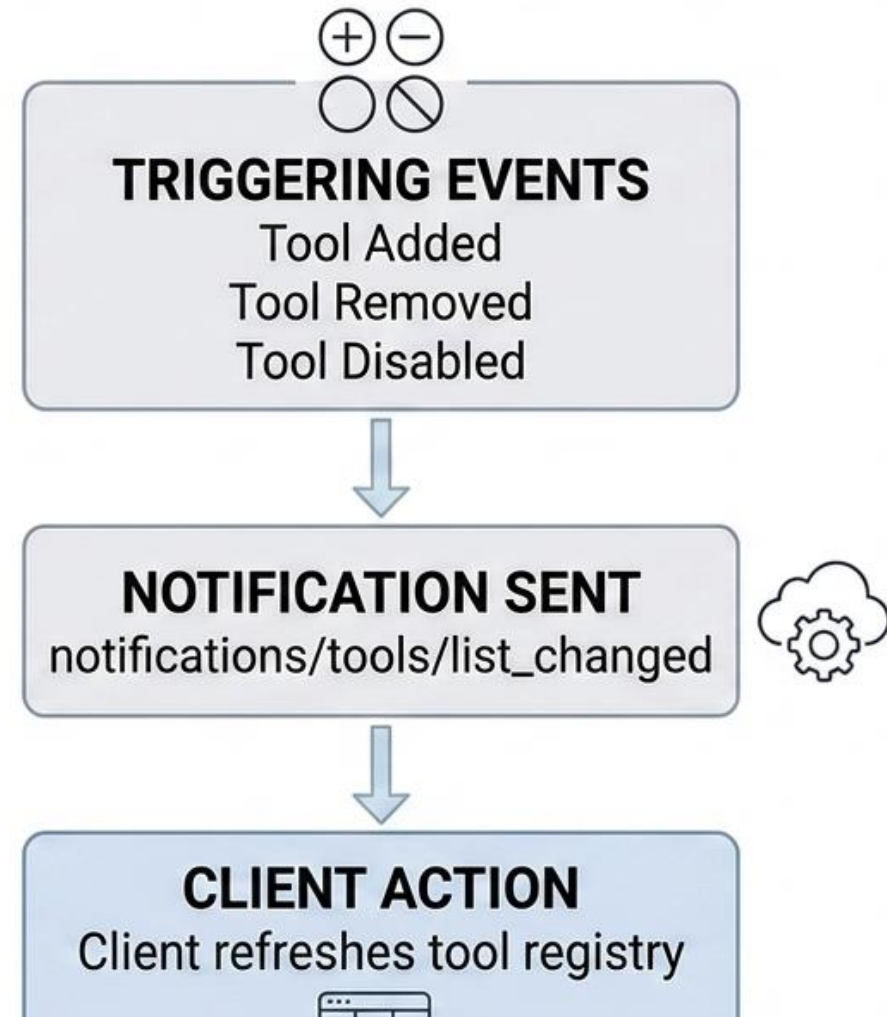
// Server -> Client: structured result, wrapped in a content array

```
{  
  "jsonrpc": "2.0", "id": 3,  
  "result": { "content": [ { "type": "text", "text": "{ \"found\": true, \"customer_id\": 482, ...}" } ] }  
}
```


Notifications

- Servers can proactively notify clients.

TOOL REGISTRY UPDATES & REFRESH



Key Property of the Data Layer

- The JSON-RPC messages **never change** based on transport.
- The protocol stays identical regardless of where the server is running.

The Transport Layer



The transport layer defines **how messages are delivered**.



Responsibilities:

Connection establishment
Message transmission
Authorization
Message framing



It **does not know anything** about:

Tools
Resources
Prompts

MCP Transport Options

Stdio	Streamable HTTP
Local child process	Remote server
Standard input/output	HTTP POST + optional SSE
No network overhead	Network communication
Single client	Multiple concurrent clients
No authentication	OAuth, API Keys, Bearer Tokens

Choosing a Transport



Studio Best for:

- Local development
- Desktop applications
- Filesystem access
- Sandboxed tools



Streamable HTTP Best for:

- SaaS integrations
- Cloud-hosted MCP servers
- Shared enterprise services
- Multiple AI agents

Why MCP?

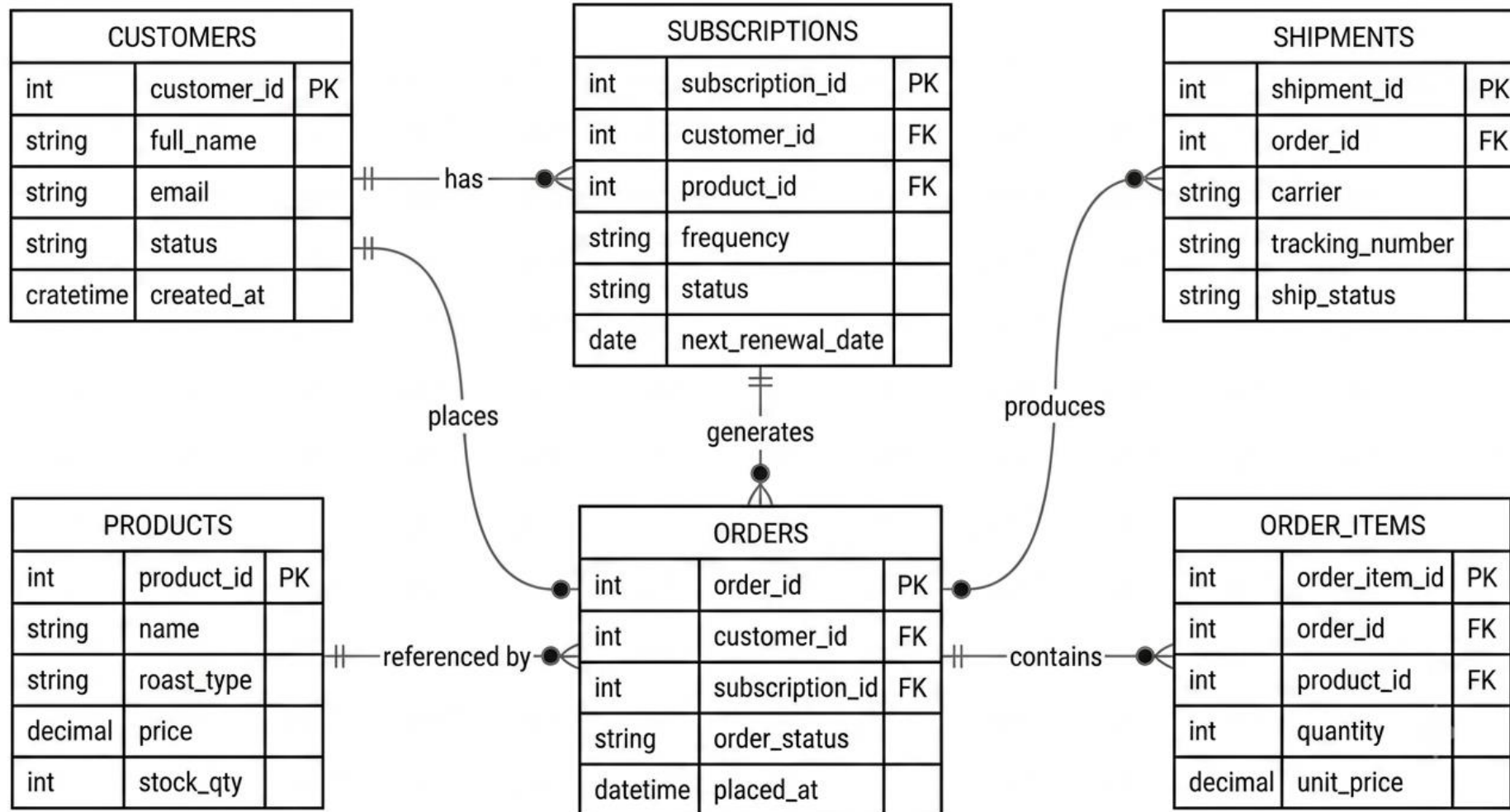
- Without MCP each agent needs custom integrations.
- While with MCP multiple agents can access your system's available resources seamlessly.
- One server per system

Example Company: BrewCraft

- **Our Running Example: BrewCraft** is a fictional coffee subscription company.
- Goal: Build an **internal AI assistant** that can:
 - Look up customer accounts
 - Check subscription status
 - View order history
 - Process simple order actions
- **Without giving the LLM direct access to the production database.**



BrewCraft Database



The Traditional Approach

- **Before MCP**
 - Many applications simply gave the model:
 - SQL access
 - Terminal access
 - Shell commands
 - The model was expected to "figure everything out."
- User → LLM → SQL / Shell Commands → Production Database

Example: Raw SQL Access

- The model generates SQL directly.
- Or even shell commands.
- Problems?
- The model is generating **arbitrary code**.

```
SELECT *  
FROM customers  
WHERE email = 'sam@example.com';
```

Why This Is Dangerous

Common production risks:

- Invalid SQL
- Hallucinated table names
- Full-table scans
- Accidental data modification
- Security vulnerabilities
- Difficult auditing

Giving an LLM unrestricted shell access is equivalent to giving it unrestricted system access.

Problems with Raw Access

Problem	Consequence
No validation	Invalid SQL and malformed commands
No least privilege	Can modify or delete data
Prompt injection	User input may manipulate commands
Poor auditing	Difficult to know what changed
Poor observability	Hard to monitor executions
Unstructured errors	Difficult for the model to recover

What Makes a Good Tool?

- Every tool has an **input schema**.
- The schema tells the model:
 - Required parameters
 - Data types
 - Allowed values
 - Parameter descriptions
- Schemas guide the model toward valid tool calls.

```
pause_subscription  
├── subscription_id : integer  
└── reason : enum
```

Writing Better Schemas

- Best practices:
 - Specify data types
 - Use enums whenever possible
 - Mark required fields
 - Disable unknown properties
 - Write descriptive field documentation
 - Use meaningful names
- Example:

Instead of: id

Prefer: subscription_id

Schema Example

- Benefits:
 - Easier for the model
 - Easier to validate
 - Fewer incorrect tool calls

```
from pydantic import BaseModel, Field
from typing import Literal

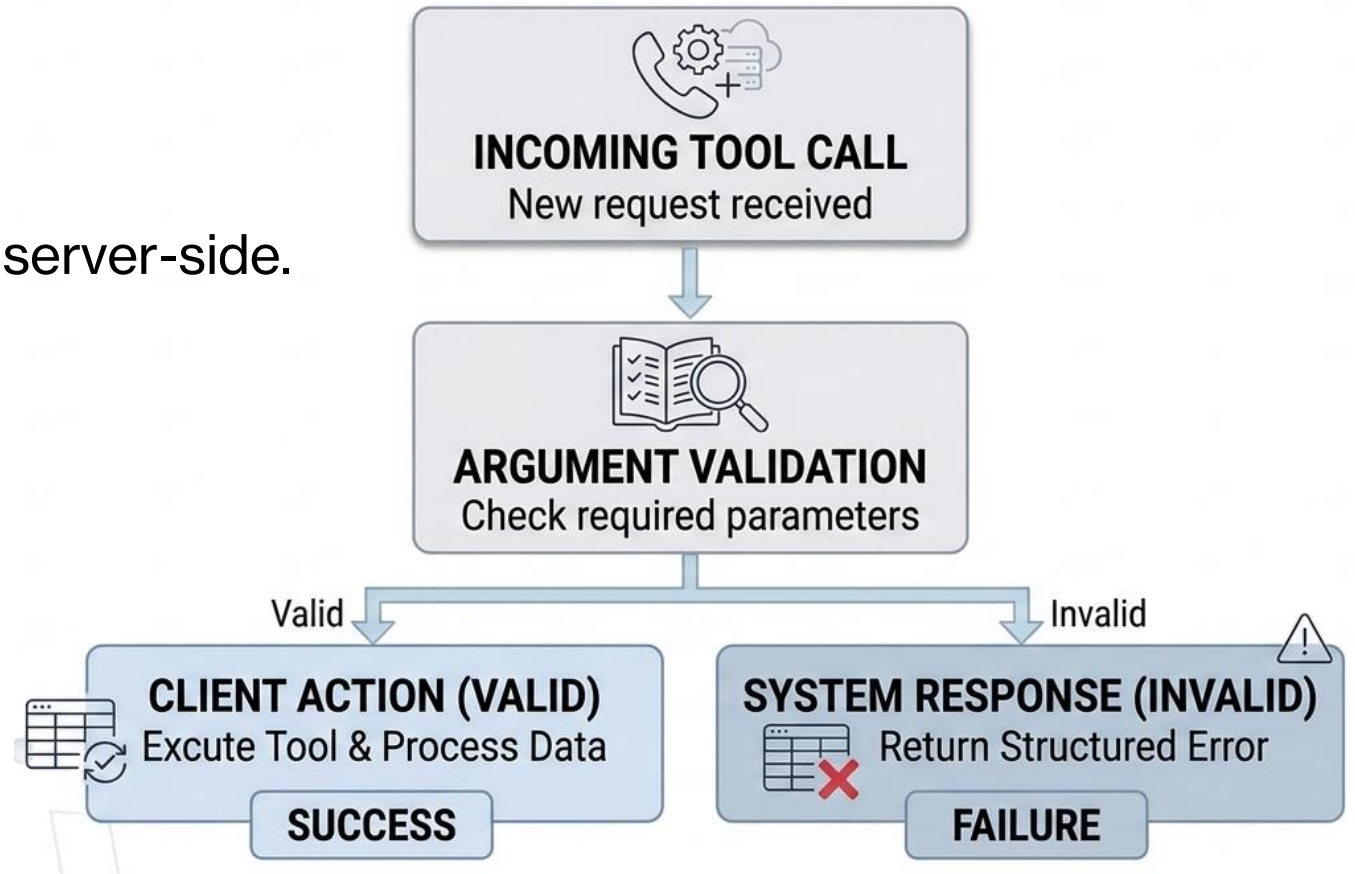
class PauseSubscriptionArgs(BaseModel):
    subscription_id: int = Field(..., description="The BrewCraft subscription ID to pause")
    reason: Literal("too_much_coffee", "traveling", "cost", "other") = Field(
        ..., description="Why the customer is pausing – used for churn analytics"
    )
```

Schema ≠ Validation

- A schema improves tool calls.
- It **does not guarantee** correctness.
- Every request must still be validated server-side.

Never trust model-generated inputs.

TOOL CALL PROCESSING & VALIDATION



Validation Example

```
from jsonschema import validate, ValidationError

def handle_tool_call(tool_schema: dict, arguments: dict):
    try:
        validate(instance=arguments, schema=tool_schema)
    except ValidationError as e:
        # Return a structured error the model can act on, not a stack trace
        return {"error": f"invalid arguments: {e.message}"}
    return execute(arguments)
```

Defensive Programming

Even valid inputs can be dangerous.

Always:

- Use parameterized SQL queries
- Check user authorization
- Require confirmation for risky actions
- Limit request sizes
- Treat tool outputs as untrusted

Golden Rule:

- The model is requesting an action not authorizing one.

Building an MCP Server



Expose a **small, well-defined set of capabilities** instead of raw database access.



Principle: Expose **business operations**, not infrastructure.



Instead of:

SQL queries
Shell commands



Expose:

Customer lookup
Subscription status
Recent orders
Pause subscription

Design the Tool Surface First

- Before writing code, decide:
- **What operations does the AI actually need?**
- Example tools:

Tool	Purpose
get_customer(email)	Find a customer
get_subscription_status(customer_id)	View subscriptions
list_recent_orders(customer_id, limit)	View order history
pause_subscription(subscription_id, reason)	Pause a subscription

Example Code

```
# brewcraft_mcp_server.py
from mcp.server.fastmcp import FastMCP
from typing import Literal

mcp = FastMCP("brewcraft")

@mcp.tool()
def get_customer(email: str) -> dict:
    """Look up a BrewCraft customer by email address."""
    # parameterized lookup against CUSTOMERS -> {found, customer_id, full_name, status, ...}

@mcp.tool()
def get_subscription_status(customer_id: int) -> list(dict):
    """Return all subscriptions and their status for a given customer."""
    # parameterized lookup against SUBSCRIPTIONS -> [{subscription_id, status, next_renewal_date, ...}]

@mcp.tool()
def list_recent_orders(customer_id: int, limit: int = 5) -> list(dict):
    """Return the most recent orders for a customer, most recent first."""
    # parameterized lookup against ORDERS -> [{order_id, order_status, placed_at}]

@mcp.tool()
def pause_subscription(
    subscription_id: int,
    reason: Literal("too_much_coffee", "traveling", "cost", "other"),
) -> dict:
    """Pause an active subscription. Requires a reason for audit logging."""
```

Creating an MCP Server

- Using the official MCP SDK:

```
mcp = FastMCP("brewcraft")
```

- Register tools using decorators:

```
@mcp.tool()  
def get_customer(...):  
    ...
```

- Each decorated function automatically becomes an MCP tool.
- Each Python function becomes, Tool name, Description, Input schema

What Happens During Inference?

Client connects to the server.

Client calls tools/list.

Server returns available tool schemas.

Model decides which tool to use.

Client sends tools/call.

Server executes the tool.

Result is returned to the model.

Model continues reasoning.

Integrating with LangChain

- langchain-mcp-adapters handles MCP automatically.

```
from langchain_mcp_adapters.client import MultiServerMCPClient
from langchain.agents import create_agent
client = MultiServerMCPClient({
    "brewcraft": {
        "command": "python",
        "args": ["brewcraft_mcp_server.py"],
        "transport": "stdio",
    }
})

tools = await client.get_tools()
agent = create_agent("anthropic:claude-sonnet-4-6", tools)
response = await agent.ainvoke({
    "messages": "Is sam@example.com's subscription active?"
})
```


MCP Decouples Components

- Any layer can be replaced independently:
 - Different LLM
 - Different MCP server
 - Different backend
- without changing the overall agent loop.

